

# CPE484 Real Time Systems - Final Coding Project

Spacecraft C&DH Programming Assignment by Austin Driggs

Homework 07 - Due 2026-04-30



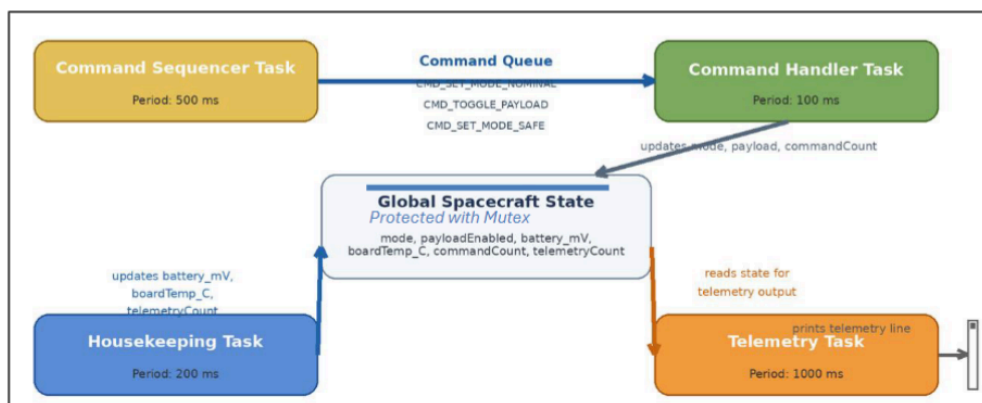
## PROMPT

In this final homework assignment, you will build a simplified spacecraft Command and Data Handling (C&DH) application in FreeRTOS. The software will model a small onboard computer that receives commands, updates spacecraft health data, and prints telemetry. You are not designing full flight computer software, instead you are implementing a compact real-time design with four periodic tasks, one command queue, and one shared spacecraft state structure.

### Project Objectives and Learning Goals

- Build a small but realistic embedded application using FreeRTOS tasks, delays, and queues.
- Practice organizing software around periodic tasks and shared system state.
- Implement one simple message-passing path using a FreeRTOS command queue.
- Apply rate-monotonic and earliest-deadline-first scheduling ideas by assigning priorities based on task periods.
- Produce clear console output that shows commands, state changes, and telemetry over time.

### Task and Data-Flow Diagram:



Submission Requirements and Deliverables:

- Your C source code and header files.
- A one-page task table listing each task, its period, and the priority you assigned for both RMS and EDF.
- A short paragraph explaining your rate-monotonic priority ordering.
- A short paragraph explaining your EDF priority ordering.
- A console log or screenshot showing the program running with RMS and EDF.

## SCHEDULING PRIORITIES

### Rate Monotonic Scheduling

The priority assignment for RMS is based on the task's period: the shorter the period (higher frequency), the higher the priority. In this C&DH architecture, the **Command Handler** and **Telemetry** tasks share the highest priority due to their 100(0) ms scaled periods. The **Command Sequencer** is assigned the lowest priority as it possesses the longest period at 500(0) ms.

| Task Name            | Period    | Priority (RMS) | Logic                                                               |
|----------------------|-----------|----------------|---------------------------------------------------------------------|
| CommandHandlerTask   | 100(0) ms | 4 (Highest)    | Tied for shortest period; must process queue data rapidly.          |
| TelemetryTask        | 100(0) ms | 4 (Highest)    | Tied for shortest period; provides high-frequency status reporting. |
| HousekeepingTask     | 200(0) ms | 2              | Moderate frequency; updates sensor data every 2 seconds.            |
| CommandSequencerTask | 500(0) ms | 1 (Lowest)     | Longest period; generates periodic background commands.             |

### Earliest Deadline First

While RMS uses static priorities, EDF is a dynamic scheduling approach where the task with the nearest absolute deadline is dynamically granted the highest priority. In a standard EDF implementation, priorities are updated dynamically at the start of each scheduler tick or task release. At  $t=0$ , the priority order often mirrors RMS because the task with the shortest relative deadline also has the closest absolute deadline. However, as the system runs, a lower-frequency task (like the Sequencer) may eventually have an absolute deadline closer than the next release of a high-frequency task, at which point its priority is temporarily boosted to ensure it completes on time.

| Task Name            | Period    | Deadline  | Priority (EDF @ $t=0$ ) |
|----------------------|-----------|-----------|-------------------------|
| TelemetryTask        | 100(0) ms | 120(0) ms | 4 (Highest)             |
| HousekeepingTask     | 200(0) ms | 210(0) ms | 3                       |
| CommandHandlerTask   | 100(0) ms | 250(0) ms | 2                       |
| CommandSequencerTask | 500(0) ms | 600(0) ms | 1 (Lowest)              |

## CODE SNIPPETS

### Task Periods

```
#define mainCMD_SEQUENCER_PERIOD    pdMS_TO_TICKS( 5000UL )
#define mainCMD_HANDLER_PERIOD      pdMS_TO_TICKS( 1000UL )
```

```
#define mainHOUSEKEEPING_PERIOD    pdMS_TO_TICKS( 2000UL )
#define mainTELEMETRY_PERIOD      pdMS_TO_TICKS( 1000UL )
```

## Data Types

```
typedef enum {
    CMD_SET_MODE_SAFE = 0,
    CMD_SET_MODE_NOMINAL = 1,
    CMD_TOGGLE_PAYLOAD = 2
} CommandType_t;

typedef enum {
    MODE_SAFE = 0,
    MODE_NOMINAL = 1
} SpacecraftMode_t;

typedef struct {
    SpacecraftMode_t mode; // Updated by CommandHandlerTask
    uint8_t payloadEnabled; // Updated by CommandHandlerTask
    uint32_t battery_mV; // Updated by Housekeeping Task
    int32_t boardTemp_C; // Updated by Housekeeping Task
    uint32_t commandCount; // Updated by CommandHandlerTask - Counter
    uint32_t telemetryCount; // Updated by Housekeeping Task - Counter
} SpacecraftState_t;

SpacecraftState_t gSpacecraft; // Global Data Store – Must be Protected with Mutex

typedef struct {
    CommandType_t command;
    uint32_t timeTag;
} CommandMessage_t;
```

## Function Prototypes

```
static void prvCommandSequencerTask( void * pvParameters );
static void prvCommandHandlerTask( void * pvParameters );
static void prvHousekeepingTask( void * pvParameters );
static void prvTelemetryTask( void * pvParameters );
```

## Command Queue and Mutex Initialization

```
/* The queue for commands. */
static QueueHandle_t xCommandQueue = NULL;

/* Mutex to protect the global spacecraft state. */
static SemaphoreHandle_t xStateMutex = NULL;
```

## Create the Queue

```
xQueue = xQueueCreate( mainQUEUE_LENGTH, sizeof( uint32_t ) ); /* Original queue */
xCommandQueue = xQueueCreate( mainQUEUE_LENGTH, sizeof( CommandMessage_t ) ); /* Command queue */
*/
xStateMutex = xSemaphoreCreateMutex(); /* State mutex */
```

## Create the Tasks

```
xTaskCreate( prvCommandHandlerTask,    "Handler",    configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 4, NULL );
xTaskCreate( prvTelemetryTask,         "Telemetry",   configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 4, NULL );
xTaskCreate( prvHousekeepingTask,      "Housekeeping", configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 2, NULL );
xTaskCreate( prvCommandSequencerTask,  "Sequencer",   configMINIMAL_STACK_SIZE, NULL,
tskIDLE_PRIORITY + 1, NULL );
```

## Functions for the Tasks

```
static void prvCommandSequencerTask( void * pvParameters )
{
    TickType_t xNextWakeTime = xTaskGetTickCount();
    const TickType_t xBlockTime = pdMS_TO_TICKS( 5000UL ); // 500ms * 10
    CommandType_t xNextCmd = CMD_SET_MODE_SAFE;

    for( ; ; )
    {
        vTaskDelayUntil( &xNextWakeTime, xBlockTime );

        /* Construct a command message with a timestamp based on system ticks. */
        CommandMessage_t xMsg;
        xMsg.command = xNextCmd;
        xMsg.timeTag = (uint32_t)xTaskGetTickCount();

        /* Send the command to the Command Handler task via the queue. */
        console_print( "--- SEQUENCER: sending %d to queue\n", xNextCmd );
        xQueueSend( xCommandQueue, &xMsg, 0U );

        /* Cycle through the available spacecraft commands in sequence. */
        if (xNextCmd >= CMD_TOGGLE_PAYLOAD) {
            xNextCmd = CMD_SET_MODE_SAFE;
        } else {
            xNextCmd++;
        }
    }
}

static void prvCommandHandlerTask( void * pvParameters )
{
    CommandMessage_t xReceivedMsg;
    TickType_t xNextWakeTime = xTaskGetTickCount();

    for( ; ; )
    {
        vTaskDelayUntil( &xNextWakeTime, mainCMD_HANDLER_PERIOD );

        /* Check if a command is received from the sequencer. */
        if( xQueueReceive( xCommandQueue, &xReceivedMsg, 0U ) == pdPASS )
        {
            xSemaphoreTake( xStateMutex, portMAX_DELAY );
            {
                /* Modify the global spacecraft state based on the specific command. */
            }
        }
    }
}
```

```

        switch( xReceivedMsg.command )
        {
            case CMD_SET_MODE_SAFE:
                gSpacecraft.mode = MODE_SAFE;
                break;
            case CMD_SET_MODE_NOMINAL:
                gSpacecraft.mode = MODE_NOMINAL;
                break;
            case CMD_TOGGLE_PAYLOAD:
                gSpacecraft.payloadEnabled = !gSpacecraft.payloadEnabled;
                break;
        }
        gSpacecraft.commandCount++;

        /* Log details of the executed command to the console. */
        console_print( "--- HANDLER: execute %d - mode: %d - payload: %d\n",
                        xReceivedMsg.command, gSpacecraft.mode, gSpacecraft.payloadEnabled
);
    }
    xSemaphoreGive( xStateMutex );
}
}

static void prvHousekeepingTask( void * pvParameters )
{
    TickType_t xNextWakeTime = xTaskGetTickCount();
    const TickType_t xBlockTime = pdMS_TO_TICKS( 2000UL ); // 200ms * 10

    for( ; ; )
    {
        vTaskDelayUntil( &xNextWakeTime, xBlockTime );

        xSemaphoreTake( xStateMutex, portMAX_DELAY );
        {
            /* Update global health and increment the telemetry counter. */
            gSpacecraft.battery_mV = ( rand() % 250 ) + 1; // 1 to 250mV
            gSpacecraft.boardTemp_C = ( rand() % 101 ) - 50; // -50 to 50 degC
        }
        xSemaphoreGive( xStateMutex );
    }
}

static void prvTelemetryTask( void * pvParameters )
{
    TickType_t xNextWakeTime = xTaskGetTickCount();
    const TickType_t xBlockTime = pdMS_TO_TICKS( 1000UL ); // 100ms * 10

    for( ; ; )
    {
        vTaskDelayUntil( &xNextWakeTime, xBlockTime );

        xSemaphoreTake( xStateMutex, portMAX_DELAY );
        {
            gSpacecraft.telemetryCount++;

```

```

/* Log of current spacecraft telemetry. */
console_print( "--- TELEMETRY (TICK: %u - COUNT: %u) --- MODE: %s --- PLD: %s --- BATT:
%u mV --- TEMP: %d C --- CMD_COUNT: %u\n",
    (uint32_t)xTaskGetTickCount(), /* Current tick count */
    gSpacecraft.telemetryCount,
    (gSpacecraft.mode == MODE_NOMINAL) ? "NOMINAL" : "SAFE",
    (gSpacecraft.payloadEnabled) ? "ON" : "OFF",
    gSpacecraft.battery_mV,
    gSpacecraft.boardTemp_C,
    gSpacecraft.commandCount );
}
xSemaphoreGive( xStateMutex );
}
}

```

## RUNNING

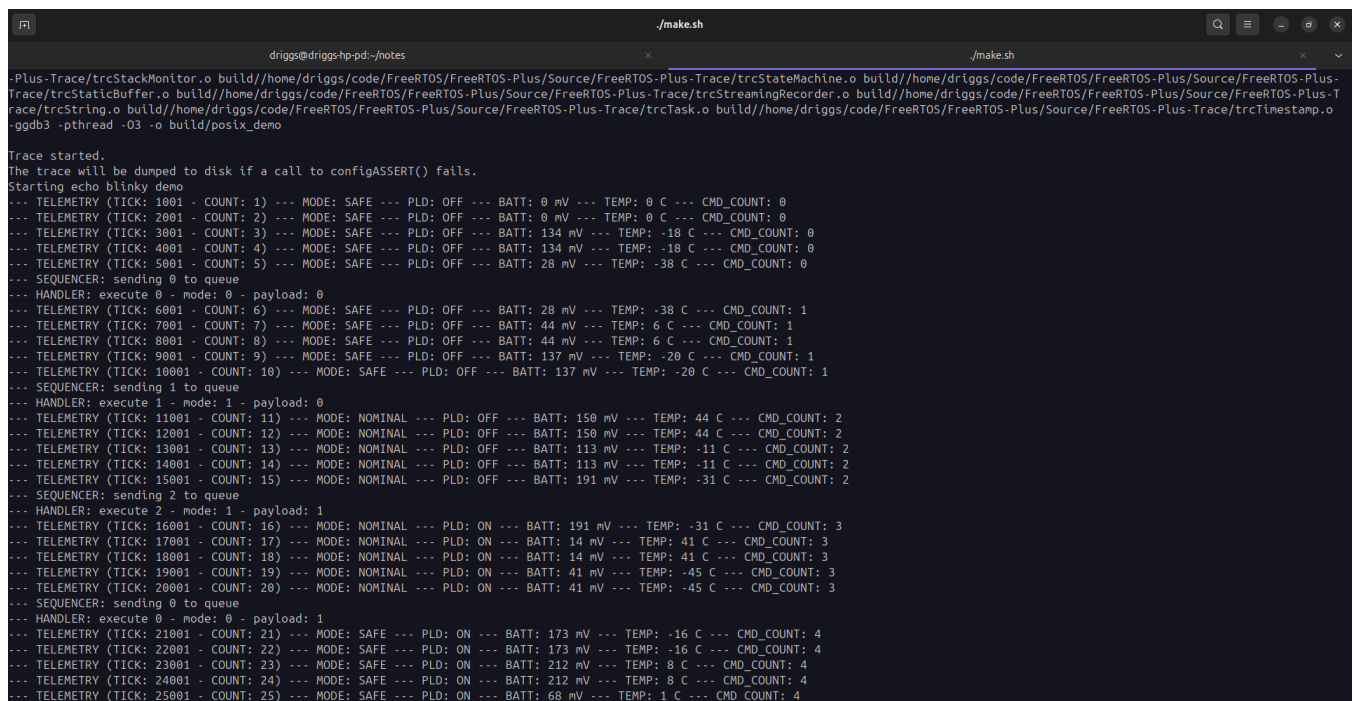
I ran the following commands:

```

cd ~/code/FreeRTOS/FreeRTOS/Demo/Posix_GCC
make clean
make CFLAGS="-DUSER_DEMO=0"
./build/posix_demo

```

And got this output:



```

driggs@driggs-hp-pd:~/notes
./make.sh

-Plus-Trace/trcStackMonitor.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcStateMachine.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcStaticBuffer.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcStreamingRecorder.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcString.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcTask.o build//home/driggs/code/FreeRTOS/FreeRTOS-Plus/Source/FreeRTOS-Plus-Trace/trcTimestamp.o
-ggdb3 -pthread -O3 -o build/posix_demo

Trace started.
The trace will be dumped to disk if a call to configASSERT() fails.
Starting echo blinky demo
--- TELEMETRY (TICK: 1001 - COUNT: 1) --- MODE: SAFE --- PLD: OFF --- BATT: 0 mV --- TEMP: 0 C --- CMD_COUNT: 0
--- TELEMETRY (TICK: 2001 - COUNT: 2) --- MODE: SAFE --- PLD: OFF --- BATT: 0 mV --- TEMP: 0 C --- CMD_COUNT: 0
--- TELEMETRY (TICK: 3001 - COUNT: 3) --- MODE: SAFE --- PLD: OFF --- BATT: 134 mV --- TEMP: -18 C --- CMD_COUNT: 0
--- TELEMETRY (TICK: 4001 - COUNT: 4) --- MODE: SAFE --- PLD: OFF --- BATT: 134 mV --- TEMP: -18 C --- CMD_COUNT: 0
--- TELEMETRY (TICK: 5001 - COUNT: 5) --- MODE: SAFE --- PLD: OFF --- BATT: 28 mV --- TEMP: -38 C --- CMD_COUNT: 0
--- SEQUENCER: sending 0 to queue
--- HANDLER: execute 0 - mode: 0 - payload: 0
--- TELEMETRY (TICK: 6001 - COUNT: 6) --- MODE: SAFE --- PLD: OFF --- BATT: 28 mV --- TEMP: -38 C --- CMD_COUNT: 1
--- TELEMETRY (TICK: 7001 - COUNT: 7) --- MODE: SAFE --- PLD: OFF --- BATT: 44 mV --- TEMP: 6 C --- CMD_COUNT: 1
--- TELEMETRY (TICK: 8001 - COUNT: 8) --- MODE: SAFE --- PLD: OFF --- BATT: 44 mV --- TEMP: 6 C --- CMD_COUNT: 1
--- TELEMETRY (TICK: 9001 - COUNT: 9) --- MODE: SAFE --- PLD: OFF --- BATT: 137 mV --- TEMP: -20 C --- CMD_COUNT: 1
--- TELEMETRY (TICK: 10001 - COUNT: 10) --- MODE: SAFE --- PLD: OFF --- BATT: 137 mV --- TEMP: -20 C --- CMD_COUNT: 1
--- SEQUENCER: sending 1 to queue
--- HANDLER: execute 1 - mode: 1 - payload: 0
--- TELEMETRY (TICK: 11001 - COUNT: 11) --- MODE: NOMINAL --- PLD: OFF --- BATT: 150 mV --- TEMP: 44 C --- CMD_COUNT: 2
--- TELEMETRY (TICK: 12001 - COUNT: 12) --- MODE: NOMINAL --- PLD: OFF --- BATT: 150 mV --- TEMP: 44 C --- CMD_COUNT: 2
--- TELEMETRY (TICK: 13001 - COUNT: 13) --- MODE: NOMINAL --- PLD: OFF --- BATT: 113 mV --- TEMP: -11 C --- CMD_COUNT: 2
--- TELEMETRY (TICK: 14001 - COUNT: 14) --- MODE: NOMINAL --- PLD: OFF --- BATT: 113 mV --- TEMP: -11 C --- CMD_COUNT: 2
--- TELEMETRY (TICK: 15001 - COUNT: 15) --- MODE: NOMINAL --- PLD: OFF --- BATT: 191 mV --- TEMP: -31 C --- CMD_COUNT: 2
--- SEQUENCER: sending 2 to queue
--- HANDLER: execute 2 - mode: 1 - payload: 1
--- TELEMETRY (TICK: 16001 - COUNT: 16) --- MODE: NOMINAL --- PLD: ON --- BATT: 191 mV --- TEMP: -31 C --- CMD_COUNT: 3
--- TELEMETRY (TICK: 17001 - COUNT: 17) --- MODE: NOMINAL --- PLD: ON --- BATT: 14 mV --- TEMP: 41 C --- CMD_COUNT: 3
--- TELEMETRY (TICK: 18001 - COUNT: 18) --- MODE: NOMINAL --- PLD: ON --- BATT: 14 mV --- TEMP: 41 C --- CMD_COUNT: 3
--- TELEMETRY (TICK: 19001 - COUNT: 19) --- MODE: NOMINAL --- PLD: ON --- BATT: 41 mV --- TEMP: -45 C --- CMD_COUNT: 3
--- TELEMETRY (TICK: 20001 - COUNT: 20) --- MODE: NOMINAL --- PLD: ON --- BATT: 41 mV --- TEMP: -45 C --- CMD_COUNT: 3
--- SEQUENCER: sending 0 to queue
--- HANDLER: execute 0 - mode: 0 - payload: 1
--- TELEMETRY (TICK: 21001 - COUNT: 21) --- MODE: SAFE --- PLD: ON --- BATT: 173 mV --- TEMP: -16 C --- CMD_COUNT: 4
--- TELEMETRY (TICK: 22001 - COUNT: 22) --- MODE: SAFE --- PLD: ON --- BATT: 173 mV --- TEMP: -16 C --- CMD_COUNT: 4
--- TELEMETRY (TICK: 23001 - COUNT: 23) --- MODE: SAFE --- PLD: ON --- BATT: 212 mV --- TEMP: 8 C --- CMD_COUNT: 4
--- TELEMETRY (TICK: 24001 - COUNT: 24) --- MODE: SAFE --- PLD: ON --- BATT: 212 mV --- TEMP: 8 C --- CMD_COUNT: 4
--- TELEMETRY (TICK: 25001 - COUNT: 25) --- MODE: SAFE --- PLD: ON --- BATT: 68 mV --- TEMP: 1 C --- CMD_COUNT: 4

```